

aftermediate

your post-FSc compass

AI-driven, hyper-localised career counselling and university entry-roadmap platform for Pakistani FSc, ICS, I.Com and A-Level students.

Document type	Product & Technical Design Document (PDD)
Version	1.0
Status	Implementation baseline — approved for build
Prepared by	Product & Engineering
Repository	github.com/maaaz52/aftermediate
Date	26 August 2026
Classification	Confidential — internal

Document Control

Revision history, ownership and review plan for this document.

1. Revision History

Version	Date	Author	Change Summary
0.1	22 Aug 2026	Engineering	Initial skeleton — problem space, goals, stack proposal.
0.2	25 Aug 2026	Engineering	Data model, aggregate-formula spec, landing page design.
0.3	26 Aug 2026	Engineering	Onboarding-quiz design spec and implementation plan merged.
1.0	26 Aug 2026	Product & Engineering	Baseline — full product + technical reference. Approved to build.

2. Document Purpose

This document is the single source of truth for the **aftermediate** product during its build phase. It combines a **Product Requirements Document** (what we are building and why) with a **Technical Design Document** (how it is built). Engineers, designers, stakeholders and the AI-assistants contributing to the codebase should read this before writing code, and should keep it in sync as the product evolves.

3. Review & Approval

Role	Name / Team	Sign-off
Product Owner	aftermediate product team	Approved v1.0
Engineering Lead	Engineering	Approved v1.0
Design	Design (see Design System)	Approved v1.0

4. Related Documents

- Spec — docs/superpowers/specs/2026-08-26-onboarding-quiz-design.md
- Plan — docs/superpowers/plans/2026-08-26-onboarding-quiz.md
- Schema — supabase/schema.sql

How to use this document. New engineers and AI assistants must read §5–§8 before writing code, §10–§12 before touching data or APIs, and §14 before changing UI. When a feature changes, update the relevant section in the same commit.

Table of Contents

1	Executive Summary
2	Problem & Opportunity
	2.1 The post-FSc decision crisis
	2.2 The information gap
3	Product Vision & Goals
4	Target Users & Personas
5	Scope
	5.1 In scope (v1)
	5.2 Out of scope (v1)
6	Functional Requirements
	6.1 Public landing & auth
	6.2 Onboarding quiz (7 sections)
	6.3 Dashboard
	6.4 Profile
	6.5 Merit engine
	6.6 Career explorer
	6.7 Trends
	6.8 Money & scholarships
	6.9 Convince (parent report)
	6.10 Ustaad — study assistant
	6.11 Rahbar — site assistant
7	Non-Functional Requirements
8	Success Metrics
9	Tech Stack
10	System Architecture
11	Data Model
12	API Specification
13	AI & Prompt Design
14	Design System & UI/UX

- 15 Aggregate Mathematics**
- 16 Security & Privacy**
- 17 Deployment & Environments**
- 18 Testing Strategy**
- 19 Roadmap & Future Work**
- 20 Risks & Mitigations**

1. Executive Summary

aftermediate is a purpose-built, AI-assisted career-counseling platform for Pakistani intermediate-level students. Every year, hundreds of thousands of FSc, ICS, I.Com and A-Level students finish school with almost no reliable, individualised guidance about what to do next. Decisions are made on rumour, coaching-academy marketing, and parental pressure — with life-long financial consequences.

The product closes that gap with a single, honest, data-backed path:

1. **Collect** the student's real position — stream, marks, entry-test plans, budget, parental pressure, skills — via a short, resumable 7-section onboarding quiz.
2. **Compute** real merit numbers using the actual university aggregate formulas (NUST, FAST, PMDC/MDCAT, UET), so the student sees their true standing instead of an academy's marketing number.
3. **Counsel** with an AI guide (Rahbar) and an AI study tutor (Ustaad), both grounded in Pakistani data, plus real labour-market trends and affordability analysis.
4. **Convince** the people who actually control the decision — parents — with a bilingual, printable, data-backed report.

The v1 product is a fully functional web application: public landing page, Supabase authentication, a complete onboarding quiz with OCR marksheet scanning, eight authenticated feature pages, three server-side AI API routes, a production coming-soon gate, and a 28-case automated test suite. It is deployed on Vercel and versioned on GitHub.

Headline metric for v1. A student who completes the quiz learns their real NUST / medical aggregate instead of a score computed from a zero (see §15 — this was the platform's original data bug). This single fix changes the merit page from misleading to honest.

2. Problem & Opportunity

2.1 The post-FSc decision crisis

Pakistan's intermediate (FSc) cohort is among the largest in the world. The numbers that frame the decision are stark:

Statistic	Value	Implication
MDCAT candidates vs MBBS seats	~180,000 candidates vs ~11,000 seats	Roughly 1 in 15 pre-medical students gets a seat — most need a real Plan B.
Youth (15–24) unemployment	~12.6% (PBS LFS 2024–25)	A wrong degree choice has an immediate, measurable cost.
IT exports	US\$4.6B (FY26), targeting US\$15B by 2030	Technology is the fastest-growing escape route for non-medical students.
NUST merit weight of entry test	75% NET	Entry-test strategy matters more than marksheet — almost nobody is told this.

2.2 The information gap

Students have access to plenty of information (merit lists, academy pamphlets, forums) but almost no counseling. The gap has four concrete causes, all of which v1 addresses directly:

1. **Aggregates are never computed honestly.** Coaching academies quote inflated "closing merits". The real formulas (NUST 75/15/10, FAST 50/40/10, PMDC 50/40/10) are public but never applied to an individual's own marks. §15 documents the exact formulas.
2. **Money is never treated as a merit factor.** Affordability, scholarships, and relocation ability change which institutions are realistic — but they're never collected, so never factored in.
3. **Parental pressure is ignored by every tool.** The person making the final decision is usually the parent, and parents are persuaded by data, not by the student's enthusiasm. No existing tool produces a parent-facing argument.
4. **No one collects the student's own data.** Before aftermediate, the platform's own pages read profile fields that nothing ever populated — the merit page computed aggregates from a default score of zero, and the "convince" page reset its inputs on every page load.

Market fit signal. The product is deliberately hyper-local: it speaks Urdu where it matters, uses Pakistani boards, cities, PKR budgets, and real university entry tests. Generic international career sites cannot provide this density of local relevance.

3. Product Vision & Goals

3.1 Vision

Every student in Pakistan who finishes intermediate school should be able to make their next decision with the same quality of data a paid career counselor would give them — for free, on their phone.

3.2 Strategic Goals (v1)

#	Goal	How it shows up in the product
G1	Collect the student's real position once, and reuse it everywhere	Single onboarding quiz feeds merit, money, convince, profile, career pages.
G2	Make merit honest	Real aggregate formulas applied to real marks; entry-test scores collected and used (never defaulted to zero).
G3	Give students a voice with their parents	Bilingual, printable parent report built from the student's real data.
G4	Guide, don't lecture	Two AI assistants with distinct, hard-scoped roles (site guidance vs study tutoring).
G5	Resume, don't restart	Quiz progress survives tab-close and device switch (localStorage + Supabase).

3.3 Guiding Principles

- **Honesty over optimism.** A hard truth ("your aggregate puts you in the top 25%") beats a flattering lie. Data is sourced and citable.
- **Local by default.** Urdu where natural, PKR budgets, Pakistani boards and institutions, realistic scholarship info.
- **One source of truth.** The quiz is the canonical home of student data; everything else reads it. Avoid duplicate state.
- **Data-driven UX.** Questions and recommendations are data, not hardcoded JSX, so content changes are edits not surgery.

4. Target Users & Personas

Persona A — "Ayesha" · Pre-Medical, 17

Context: Lahore, FSc Pre-Medical, scored ~93% FSc. Parents assume she will "do MBBS". No realistic idea of the 1-in-15 odds.

Needs: to know her real medical aggregate, her realistic alternatives, and a way to show her parents the data.

Path through product: onboard → merit (MDCAT aggregate + percentile) → trends / career for Plan B fields → convince (parent report).

Persona B — "Bilal" · Pre-Engineering, 18

Context: Karachi, FSc Pre-Engineering. Considering NUST / FAST. Zero coaching budget. Hears "everything is possible" from everyone.

Needs: exact NET/FAST aggregate, entry-test strategy, affordable institution options, scholarship help.

Path: onboard → merit (NUST + FAST) → money (budget tiers) → Ustaad (NET prep).

Persona C — "Zara" · ICS, 17

Context: Islamabad, ICS. Parents expect "something in government". Zara likes coding but hasn't told them.

Needs: a credible tech career path, salary/trend data, and a gentle way to raise it at home.

Path: onboard → trends (IT exports) → career (tech majors) → convince.

Persona D — "Hamza" · I.Com / A-Level, 18

Context: Multan, considering business abroad. Needs a Plan A and a Plan B, cost-aware.

Needs: affordability analysis, abroad options, scholarship eligibility.

Path: onboard → money (scholarships, budget) → trends (abroad) → Ustaad.

Non-target users (v1). University graduates, working professionals, and students in the early years of FSc are out of scope; the platform is optimised for the final-year decision window.

5. Scope

5.1 In Scope (v1)

Area	Delivered
Public site	Landing page with stats, features, how-it-works, reality checks and sources; production / coming-soon gate for non-landing routes.
Identity	Supabase email/password + Google OAuth; session persistence; role-light (no admin roles in v1).
Onboarding	7-section resumable quiz, OCR marksheet scanning, conditional questions, progress bar, completion gating of the app.
Core pages	Dashboard, Profile (editable), Merit engine, Career explorer, Trends, Money & scholarships, Convince (parent report), Study (Ustaad).
AI assistants	Rahbar (site guide, slide-in drawer) and Ustaad (study tutor, full page).
Data	Static JSON knowledge base (universities, majors, scholarships, courses, reality checks, jobs, industry, abroad).
Testing	Vitest unit suite for pure logic (quiz completeness, aggregates, profile merge).

5.2 Out of Scope (v1)

- Payment / paid tiers; marketing analytics dashboards.
- Admin/content-management UI (data edits happen in the JSON files / Supabase console).
- Question-level history or versioning of quiz answers.
- Mobile native apps (responsive web only).
- Real-time chat with human counselors.
- Automated OCR batching / bulk marksheet upload.

6. Functional Requirements

Each section lists the page's purpose, key user stories, and acceptance criteria. Reference implementations live in `src/app`; the canonical data model is §11.

6.1 Public Landing & Auth

Landing page — `/`

- Hero with the product promise and a CTA into the onboarding flow.
- Sections: `#explore`, `#features`, `#how`, `#momentum` (marquee), `#reality` (reality-check cards), `#sources`.
- Public in all environments.

Login — `/login` and auth dialog

- Email/password sign-in and sign-up (with optional name/stream metadata).
- Google OAuth with redirect back to `/dashboard`.
- After successful auth, redirect to `/dashboard`; the app layout gate bounces to `/onboard` if the quiz is incomplete.
- Server-side API routes are exempt from the auth gate; they validate themselves.

6.2 Onboarding Quiz — `/onboard`

The quiz is a 7-section, data-driven flow. Sections and required answers:

#	Section	Questions	Required
0	Stream	stream (pre-medical / pre-engineering / ICS / I.Com / A-Levels)	stream
1	Marksheet	OCR upload; matric & FSc obtained/total; FSc Part-1 (only if NET)	matric + FSc totals
2	Entry test	board, exam year, entry test (NET/MDCAT/ECAT/none), score	entry test
3	Money	city, province, monthly budget (PKR), relocation, scholarship need	city, budget
4	Pressure	dream field, parents' expectation, decision maker, parents' firmness (1-5)	parents' expectation, decision maker
5	Readiness	certifications, projects, English (1-5), consistency (1-5)	English, consistency
6	Interests	multi-select interests	at least 1

- **Questions are data** (`src/lib/quiz.ts`): id is a dotted path (`quiz.city`, `marks.fsc0btained`) so one setter writes anywhere without special-casing.

- **Conditional logic:** entry-test scores hidden when `entryTest = "none"`; MDCAT only for pre-medical; ECAT only for pre-engineering; Part-1 shown only for NET.
- **Resume:** progress saved per section advance to `localStorage`; `firstIncompleteSection()` is the cross-device fallback; stale saved indices can never skip an unfinished section.
- **Gating:** `(app)/layout.tsx` redirects signed-in users with an incomplete quiz to `/onboard` — this covers direct URL navigation, not just the login path. Hydration must complete before gating to avoid bouncing completed users.

6.3 Dashboard — `/dashboard`

- Greeting ("Salam, [name]."), stream badge, FSc percentage badge.
- Reality-check card for the user's stream.
- Aggregate cards (NUST / FAST / MDCAT depending on stream) with links to the merit page.
- "Fields that fit your stream" (up to 6 majors) with salary ranges and demand badges.
- Shortcut cards to Merit / Career / Money / Convince.
- "Talk to Rahbar" CTA (opens the slide-in drawer) and links to profile editing.

6.4 Profile — `/profile`

- Hero: avatar upload (client-side, 2 MB image limit, data-URL stored), name and bio inline-editing, email, quiz-completion badge.
- Stat strip: FSc %, skills count, interests count.
- Editable sections: **Academics** (stream, board, year, entry test, marks), **Education** (add/remove rows: degree, institution, year, grade), **Skills** (40 toggleable chips), **Interests** (toggleable chips), **Money & location, Decisions & pressure**.
- Every section uses a consistent Edit → inline form → Save/Cancel pattern.
- "Edit quiz →" links back to `/onboard`.

6.5 Merit Engine — `/merit`

- Computes and explains NUST, FAST (computing and engineering splits), and PMDC/MDCAT aggregates.
- Interactive: any number can be adjusted and the aggregate recomputes live.
- Shows the exact weight breakdown per university (see §15).
- MDCAT percentile + "band" label (e.g. "Top 10%").
- Data source links for every figure.

6.6 Career Explorer — `/career` and `/career/[id]`

- List of majors filtered to the student's stream, with demand badges and salary ranges.
- Detail page per major: description, why-now stat, skill tree, day-in-the-life simulator (scenario + choices + results), courses, universities.
- Static data-backed; no AI required at view time.

6.7 Trends — `/trends`

- Labour-market and industry data: jobs, industry exports, abroad destinations.

- Charts (Recharts) and citable sources.

6.8 Money & Scholarships — /money

- Reads the quiz's monthly budget (`quiz.budgetMonthly`) and computes affordability tiers at PKR 20k / 60k / 150k thresholds.
- Scholarship directory (HEC, Fulbright, etc.) with eligibility and deadlines.

6.9 Convince (Parent Report) — /convince

- A bilingual (English/Urdu) printable report for parents.
- Seeded from the student's real profile data (replaces the old hardcoded `0,0,3,3` state).
- Includes reality-check stats, a career-data chart, and a parent-facing takeaway in Urdu.
- Print/PDF export via the browser.

6.10 Ustaad — Study Assistant — /study

- Full-page AI tutor chat for FSc / MDCAT / NET / ECAT study help.
- Persona-scoped prompt (see §13); explicitly **not** a general assistant.
- Suggestion chips ("Make me a 2-week MDCAT revision plan", ...), streaming responses, `localStorage` history.

6.11 Rahbar — Site Assistant (drawer)

- Slide-in drawer from the right edge on any app page; opened from the sidebar "Talk to Rahbar" button or the dashboard CTA.
- Persona-scoped to **site guidance** — how to use pages, what features do, where data lives. Redirects study questions to Ustaad.
- Backdrop + close, streaming responses, `localStorage` history, suggestion chips.

7. Non-Functional Requirements

Category	Requirement	How it's met
Performance	Pages feel instant; landing is static/ prerendered.	Next.js App Router; static generation for all content pages; client-side data only where needed.
Reliability	No lock-out: users must never be trapped in the quiz.	Completeness rules are unit-tested; hydration is awaited before gating.
Persistence	Progress survives tab close and device switch.	localStorage cache + debounced Supabase upsert; last-write-wins on <code>updated_at</code> .
Security	Users access only their own data.	Supabase RLS on every table; service-role key server-side only.
Accessibility	Keyboard-navigable inputs, focus rings, aria-labels on icon buttons.	Design-system focus states; labelled controls.
Responsive	Usable from phone to desktop.	Mobile-first Tailwind; sidebar collapses to a top bar on small screens.
Budget	API cost kept low.	Streaming responses; lightweight <code>gemini-flash</code> model; client-side history.
Observability	Failures are visible.	Console-warned sync failures; structured chat error responses.

8. Success Metrics

Metric	v1 Target	Why it matters
Quiz completion rate	$\geq$ 50% of users who start finish	Downstream pages depend on quiz data.
Aggregate accuracy	100% of users see a non-zero, formula-correct aggregate when data present	Correctness is the product's core promise (\$15).
Resume success	Returning users resume at the right section	Validates the resume/`firstIncompleteSection` design.
Ustaad / Rahbar engagement	$\geq$ 1 message sent per session on average	AI assistants are the retention layer.
Parent report generated	$\geq$ 20% of completed users	The convincing of parents is the real conversion moment.

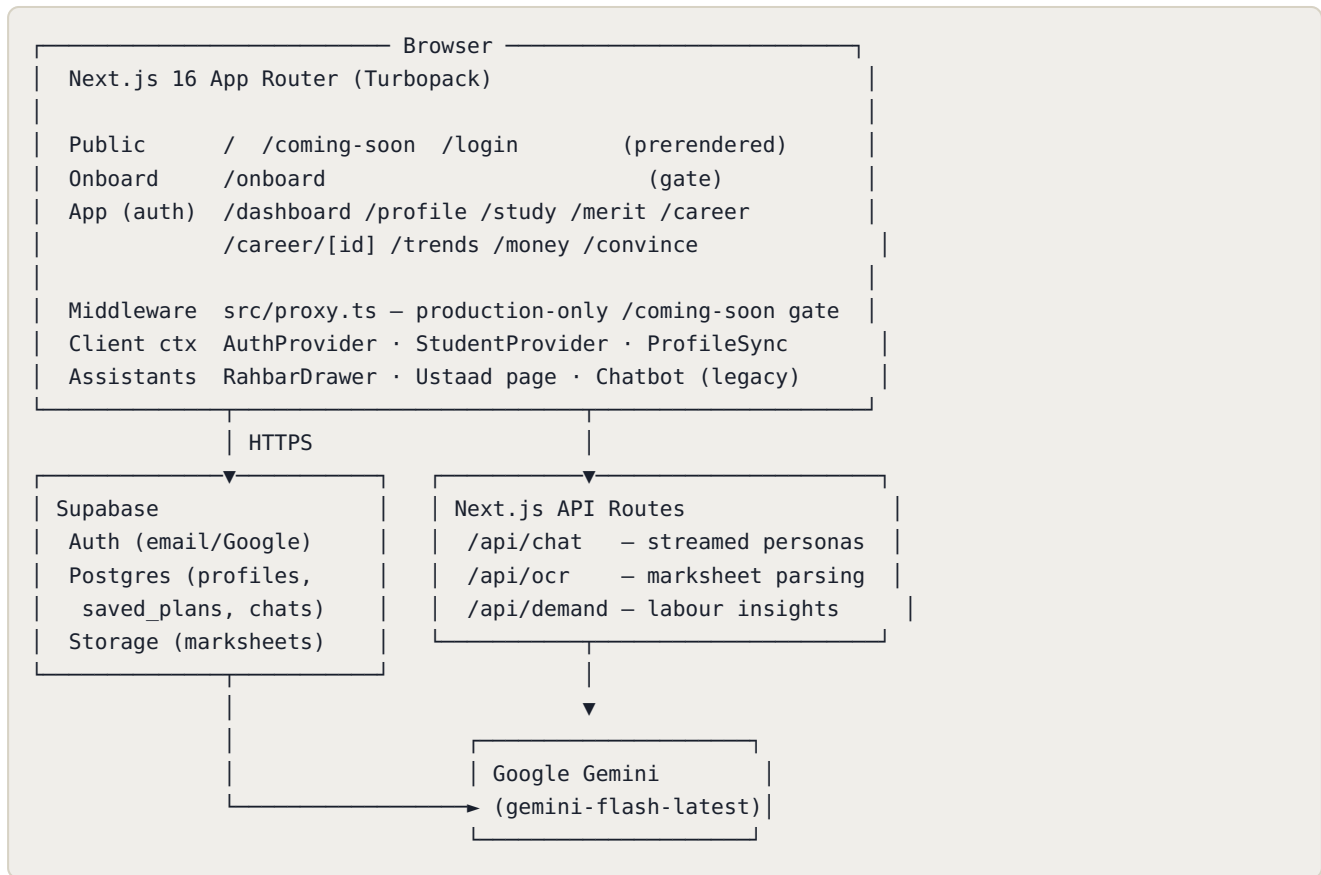
9. Tech Stack

Layer	Technology	Version	Role
Framework	Next.js (App Router, Turbopack)	16.3.2	SSR/SSG, routing, middleware, API routes.
Language	TypeScript	^5	Typed application code.
UI	React + Tailwind CSS v4	19.2.8 / ^4	Component UI and design tokens.
Components	shadcn/ui + lucide-react + CVA	—	Design-system primitives and icons.
Backend / DB	Supabase (Postgres + Auth + Storage)	^2.112.3	Profiles, saved plans, chat sessions, auth, object storage.
AI	Vercel AI SDK + Google Gemini	^7.0.77 / @ai-sdk/google	Streaming chat, OCR, structured generation.
Charts	Recharts	^3.10.1	Trend and parent-report charts.
Tests	Vitest	^4.1.11	Unit tests for pure logic.
Deploy	Vercel + GitHub	—	CI/deploy; git host.
Fonts	@fontsource (Silkscreen, Plus Jakarta Sans, Space Mono)	^5.3.0	Self-hosted display/sans/mono.

Note on AI model. The default model is `gemini-flash-latest`, overridable via `G00GLE_MODEL`. Flash is chosen for cost and speed at FSc-grade question depth.

10. System Architecture

10.1 High-Level View



10.2 Client-Side State & Sync

Student data lives in a single React context (`src/lib/store.tsx`) backed by `localStorage`. A global `ProfileSync` controller (mounted in the root layout) owns the two-way sync:

1. **Hydrate (remote → local):** on sign-in, fetch the user's `profiles` row once. If the remote `updated_at` is newer than the local stamp, replace the store from remote; otherwise keep local.
2. **Push (local → remote):** debounced ~800 ms upsert on every profile change (avoids the write storm of per-keystroke writes).
3. **Conflict rule:** last-write-wins on `updated_at`. A single student on their own devices — acceptable.
4. **quizStep is intentionally not synced;** cross-device resume uses the computed `firstIncompleteSection`.

10.3 Middleware / Routing Gate

`src/proxy.ts` runs on every request. In **production**, any path that is not `/` or `/coming-soon` is redirected to `/coming-soon` — this gates the public demo. In **development**, all pages are accessible so changes can be previewed. API routes are excluded from the matcher.

Operational caveat. Because the production gate sends the public demo to `/coming-soon`, the full app flow is only reachable in dev (or by relaxing the gate). A Vercel preview-deployment toggle is on the roadmap (\$19).

10.4 Folder Map

```
src/  
  app/  
    layout.tsx      Root layout – providers + ProfileSync  
    page.tsx        Landing  
    proxy.ts        (Next middleware) coming-soon gate  
    (app)/          Authenticated area (layout gate + sidebar + top nav)  
      layout.tsx     Quiz-completion gate  
      dashboard/ profile/ study/ merit/ career/ trends/ money/ convince/  
      career/[id]/   Major detail  
      login/ onboard/ coming-soon/  
      api/chat/ api/ocr/ api/demand/  
  components/       Sidebar, TopNav, RahbarDrawer, ProfileSync,  
                    quiz/* (question renderers), ui/* (shadcn)  
  lib/              store.tsx, quiz.ts, aggregates.ts, ai.ts,  
                    data.ts, merge-profile.ts, auth.tsx, types.ts  
  data/             Static JSON knowledge base
```

11. Data Model

11.1 `profiles` (one row per authenticated user)

Column	Type	Notes
id	uuid (PK, FK → auth.users)	Cascade delete.
name	text	Defaulted from sign-up metadata.
avatar	text	Data URL (client-side upload).
bio	text	Short personal bio.
stream	text (check constraint)	pre-medical · pre-engineering · ics · icom · alevel.
board	text	Mirrors quiz board.
marks	jsonb	Matric/FSc/Part-1/entry-test marks (§11.3).
interests	text[]	Multi-select interests.
skills	text[]	Profile skill chips.
education	jsonb	Array of education entries.
city / budget	text	Denormalised mirrors of quiz.city / budgetMonthly.
quiz	jsonb	Canonical quiz answers (§11.2).
quiz_completed_at	timestampz	Null until the quiz is complete.
created_at / updated_at	timestampz	LWW conflict key.

11.2 `QuizAnswers` (jsonb, canonical)

```
{
  board?: string; examYear?: number;
  entryTest?: "net" | "mdcat" | "ecat" | "none";
  city?: string; province?: string;
  budgetMonthly?: number;           // PKR/month
  canRelocate?: "yes"|"in-province"|"no";
  needsScholarship?: "must"|"helpful"|"no";
  dreamField?: string;
  parentsExpect?: "doctor"|"engineer"|"civil-service"|"business"|"my-choice"|"unsure";
  decisionMaker?: "me"|"parents"|"together";
  parentsFirmness?: number;          // 1-5
  certifications?: number; projects?: number;
  english?: number; consistency?: number; // 1-5
}
```

11.3 Marks

```
matricObtained / matricTotal
fscObtained / fscTotal
fscPart1Obtained? / fscPart1Total?    // used by NUST formula
entryTestObtained? / entryTestTotal?  // feeds all aggregates
```

11.4 Other tables

Table	Purpose	Columns
saved_plans	User-saved roadmaps.	id, user_id, type, payload (jsonb), created_at
chat_sessions	Optional persisted AI chats.	id, user_id, persona, messages (jsonb), created_at, updated_at

11.5 Row-Level Security

All tables enable RLS. Policies restrict to the owner via `auth.uid() = id` (profiles) or `auth.uid() = user_id` (plans, chats). A `handle_new_user` trigger auto-creates the profile row on sign-up. Storage bucket `marksheets` is private with owner-scoped policies.

12. API Specification

12.1 POST /api/chat

Streams an AI response. Body: { persona, messages: {role, content}[] }. Personas: rahbar (site guide), study (Ustaad), essay, cv. Response is a text-stream (text/plain chunks). Runtime: Node, maxDuration=60.

12.2 POST /api/ocr

Parses a Pakistani FSc marksheet image (data URL) into structured marks JSON via the vision model. Strict output contract: { board, stream, matricObtained, matricTotal, fscObtained, fscTotal, fscPart1Obtained, fscPart1Total, subjects[], confidence }. Invalid/absent values are null, never guessed. Urdu digits are normalised to English.

12.3 POST /api/demand

Given { stream, interests, countries }, returns per-country labour-market insight lines via structured generation. Validates the stream against the allowed enum before calling the model.

Error convention. All API routes return 400 for invalid input and 500 for generation failures, with a JSON { error } body. Chat returns a streamed fallback string on failure rather than aborting.

13. AI & Prompt Design

All prompts live in `src/lib/ai.ts` so they are reviewable and versionable. There are four scoped personas, deliberately separated so each model call has one job.

Persona	Role	Key constraints
rahbar	Site guide (drawer)	Explains pages/features; redirects study questions to Ustaad; concise; ends with one next step.
study / "Ustaad"	FSc study tutor	Concepts, exam strategy, MDCAT/NET/ECAT prep; MDCAT weight split (Biology 45%, Chemistry 25%, Physics 20%, English 5%, LR 5%); never medical/legal advice.
essay	College/scholarship essay coach	Authentic, cliché-free personal statements; no fabricated achievements; 400–600 words + "why this works".
cv	CV/resume writer	ATS-friendly structure, action verbs, measurable results, tailoring tips.

13.1 Grounding & source discipline

- Rahbar cites sources for stats (e.g. PBS LFS 2024–25 youth unemployment 12.6%).
- Study help sticks to verifiable exam structure facts.
- Structured generation (`generateStructured`) wraps model output in a JSON parse-and-extract step; OCR and demand routes use it.
- Streaming (`streamText`) is used for chat so first tokens arrive quickly.

14. Design System & UI/UX

14.1 Visual identity

An editorial-meets-terminal aesthetic: warm paper background, ink text, a blue "saffron" accent (despite its name), pixel/offset shadows, monospace data readouts, and a grid backdrop. The brand font is Silkscreen (display) with Plus Jakarta Sans (body) and Space Mono (data).

14.2 Color tokens

Token	Hex	Use
background	#f4f2eb	App canvas
surface / surface-2	#ffffff / #ebe7db	Cards / raised fills
line	#d7d2c4	Borders
ink	#191f2c	Primary text
muted / faint	#566073 / #8a93a6	Secondary / tertiary text
accent / saffron	#2f55d4	Primary action & brand accent
emerald	#1c9e62	Positive / success
danger	#d63d3d	Errors / warnings
amber / info / violet	#d99a2b / #1790b0 / #7a5bd4	Semantic accents

14.3 Layout

- **Authenticated area:** fixed left sidebar (288px) with nav links + a "Talk to Rahbar" button; sticky top bar with brand and sign-out; content offset by the sidebar width. Sidebar is persistent and full-height.
- **Rahbar drawer:** fixed right panel (400px) that slides in over a dimmed backdrop.
- **Mobile:** responsive stacking; the sidebar collapses so content uses the full width.

14.4 Component conventions

- shadcn/ui primitives (Card, Button, Input, Badge) customised with the design tokens.
- Pixel accents: `.pixel-shadow`, `.pixel-border`, `.grid-bg`, `.dither`.
- Icons from lucide-react; every icon-only button has an `aria-label`.
- Question/quiz UI is data-driven: `QuestionKind` $\in$ `single` · `multi` · `number` · `text` · `scale` · `stream` · `marksheet`, rendered by `question-field.tsx`.

15. Aggregate Mathematics

These formulas are the product's backbone. They are implemented in `src/lib/aggregates.ts`, unit-tested, and used by the dashboard, merit page, and profile. $\text{pct} = \text{obtained} / \text{total} \times 100$.

15.1 NUST

$\text{aggregate} = \text{NET\%} \times 0.75 + \text{FSc Part-I\%} \times 0.15 + \text{Matric\%} \times 0.10$

NET total defaults to 200. Part-I is used when present, else full FSc.

15.2 FAST

Computing: $\text{test\%} \times 0.50 + \text{HSSC\%} \times 0.40 + \text{SSC\%} \times 0.10$

Engineering: $\text{test\%} \times 0.33 + \text{HSSC\%} \times 0.50 + \text{SSC\%} \times 0.17$

Entry-test total defaults to 100 for FAST.

15.3 PMDC (Medical)

$\text{aggregate} = \text{MDCAT\%} \times 0.50 + \text{HSSC\%} \times 0.40 + \text{SSC\%} \times 0.10$

MDCAT total defaults to 200. Top public colleges close at ~91–96%.

15.4 Worth Score

A readiness score used on readiness/career surfaces: $\text{FSc\%} \times .35 + \text{certs} \times 25 (\text{cap } 100) \times .25 + \text{projects} \times 25 \times .20 + \text{english} \times 20 \times .10 + \text{consistency} \times 20 \times .10$, rounded to a band: Emerging · Building momentum · High potential.

15.5 Overall Standing

Pre-test ballpark: $\text{FSc\%} \times 0.6 + \text{Matric\%} \times 0.4$, mapped to percentile bands (top 1% ... top 80%).

Why this section exists. v1's merit page originally computed every aggregate with an entry-test score of zero — NUST merit was wrong for every user. The onboarding quiz now collects the score, the formulas default totals only, and the tests pin the corrected math (\$18).

16. Security & Privacy

- **Secrets:** keys live in `.env.local` (git-ignored); `.env.example` documents names only. The Supabase `service_role` key is never used client-side. Never commit real keys.
- **RLS:** every table has owner-scoped policies; new columns are covered by existing policies.
- **Input validation:** API routes validate required fields and enums (e.g. stream) before hitting the model.
- **File uploads:** client-side images are constrained (type `image/*`, ≤ 2 MB) and stored as data URLs; marksheet uploads go to a private, owner-scoped storage bucket.
- **Prompt injection hygiene:** persona prompts are fixed server-side; user text is treated as message content, not system instructions.
- **Privacy:** no analytics SDKs in v1; chat history is localStorage/client-side; no third-party tracking.

17. Deployment & Environments

17.1 Environments

Env	Purpose	Gate behavior
Local dev	Daily build & preview	All routes accessible (no coming-soon redirect).
Vercel preview	PR review	Follows production gate unless relaxed.
Production	Public demo	Non-public routes → /coming-soon .

17.2 Required environment variables

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
GOOGLE_GENERATIVE_AI_API_KEY      # optional override: GOOGLE_MODEL
```

17.3 Release process

1. Feature branches off `main`; changes reviewed and linted (`npm run lint`), tested (`npm test`), and built (`npm run build`).
2. Merged to `main` via fast-forward; pushed to GitHub.
3. Vercel auto-deploys from `main` with the env vars above.
4. Run database migrations in Supabase SQL editor before deploy when the schema changed.

18. Testing Strategy

Unit tests (Vitest) cover the pure logic that, if wrong, locks users out or shows wrong numbers:

Suite	Covers	Count
quiz.test.ts	isQuizComplete, visibleQuestions (conditional logic), firstIncompleteSection, entry-test totals, answered-value validation (rejects seeded zero marks).	20
aggregates.test.ts	NUST/FAST/MDCAT with real and missing entry-test scores (pins the zero-score fix).	4
merge-profile.test.ts	Last-write-wins profile merge, remote/local edge cases.	4
recommend.test.ts	Recommendation logic over streams/interests.	—

Run: `npm test` (vitest run). **Quality gates:** `npm run lint` (0 errors), `npm run build` (all routes compile). Manual QA covers the quiz walk-through, resume-after-reload, gate redirect-and-release, and print/PDF of the parent report.

19. Roadmap & Future Work

Phase	Items	Priority
v1.1	Vercel preview flag to open the app on preview deploys; relaxation of the production gate behind a token.	High
v1.2	Admin/content UI for the JSON knowledge base; analytics (quiz drop-off funnels).	Medium
v1.3	Persisted chat sessions server-side (table exists); cross-device chat history.	Medium
v2.0	Human counselor handoff; verified-answer question bank for MDCAT/NET; parental notification.	Low

20. Risks & Mitigations

Risk	Impact	Mitigation
Quiz drop-off (7 sections is long)	Users never reach the app.	Resumability, progress bar, resume point; make sections 3-5 optional later if needed.
Lock-out from a buggy completeness rule	Users trapped in the quiz.	Completeness logic is unit-tested; hydration awaited before gating.
Hydration flash bouncing completed users	Completed users re-sent to quiz.	Gate waits on hydrated ; tested merge logic.
Existing users have <code>quiz_completed_at = null</code>	They are pushed through the quiz once.	Accepted; their stream/marks/interests prefill sections 0, 1, 6.
AI cost/latency	Expense and slow responses.	Flash model, streaming, client-side history.
Stale aggregate data	Wrong merit numbers.	Static JSON is versioned and sourced; formulas unit-tested.
Production gate hides the app	Demo visitors see only landing.	By design for v1; preview flag on roadmap.

aftermediate — Product & Technical Design Document · Version 1.0 · Confidential
End of document